

Artificial Intelligence

Assignment # 2 (Lab)

Name: Haroon Ishaq

Subject: Artificial Intelligence

Section: 7D

Sap ID: 70126879

Submitted to: Ma'am Aqsa

Program: BSCS (Semester 7)

Date of Submission: 17 November 2024

Implementing Iterative Deepening and Uniform Cost Search in Python

Problem Statement:

You are provided with an undirected weighted graph representing a network of cities. Your task is to:

1. Implement **Iterative Deepening Search (IDS)** to find a specific city in the graph.
2. Implement **Uniform Cost Search (UCS)** to find the shortest path between two given cities.

Graph Representation:

The graph will be represented as an adjacency list using a dictionary, where keys are city names, and values are lists of tuples representing neighboring cities and edge weights. For example:

```
graph = {  
  
    "A": [("B", 1), ("C", 4)],  
  
    "B": [("A", 1), ("D", 2), ("E", 5)],  
  
    "C": [("A", 4), ("E", 1)],  
  
    "D": [("B", 2)],  
  
    "E": [("B", 5), ("C", 1)]  
  
}
```

1. Iterative Deepening Search (IDS)

Implement IDS to search for a target city starting from a given city.

Inputs:

- `graph`: The adjacency list representing the graph.
- `start`: The starting city.
- `target`: The city to search for.

Output: A boolean indicating whether the target city was found and the path to the city if found.



Lorenz Last Checkpoint: 2 months ago

File Edit View Run Kernel Settings Help

📁 + ✂️ 📄 📄 ▶️ ⏏️ ↺ ▶️ Code ▾ 🔍

```
[2]: def iterative_deepening_search(graph, start, target):
    def dls(node, depth, path, visited):
        if depth == 0 and node == target:
            return True, path
        if depth > 0:
            for neighbor, _ in graph[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    found, p = dls(neighbor, depth - 1, path + [neighbor], visited)
                    if found:
                        return True, p
                    visited.remove(neighbor)
            return False, []

    for depth in range(len(graph)):
        visited = set([start])
        found, path = dls(start, depth, [start], visited)
        if found:
            return True, path
    return False, []

if __name__ == "__main__":
    graph = {
        "A": [("B", 1), ("C", 4)],
        "B": [("A", 1), ("D", 2), ("E", 5)],
        "C": [("A", 4), ("E", 1)],
        "D": [("B", 2)],
        "E": [("B", 5), ("C", 1)]
    }

    # Test IDS
    start_city = "A"
    target_city = "E"
    found, path = iterative_deepening_search(graph, start_city, target_city)
    print(f"IDS - Found: {found}, Path: {path}")
```

IDS - Found: True, Path: ['A', 'B', 'E']

Output:

```
IDS - Found: True, Path: ['A', 'B', 'E']
```

2. Uniform Cost Search (UCS)

Implement UCS to find the shortest path between a start city and a goal city. **Inputs:**

- `graph`: The adjacency list representing the graph.
- `start`: The starting city.
- `goal`: The destination city.

Output: The shortest path and its cost.

```
[3]: import heapq

def uniform_cost_search(graph, start, goal):
    priority_queue = [(0, start, [])] # (cost, node, path)
    visited = set()

    while priority_queue:
        cost, current, path = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)
        path = path + [current]

        if current == goal:
            return path, cost

        for neighbor, weight in graph[current]:
            if neighbor not in visited:
                heapq.heappush(priority_queue, (cost + weight, neighbor, path))

    return [], float('inf') # Return infinite cost if no path found

if __name__ == "__main__":
    graph = {
        "A": [("B", 1), ("C", 4)],
        "B": [("A", 1), ("D", 2), ("E", 5)],
        "C": [("A", 4), ("E", 1)],
        "D": [("B", 2)],
        "E": [("B", 5), ("C", 1)]
    }

    # Test UCS
    start_city = "A"
    goal_city = "E"
    path, cost = uniform_cost_search(graph, start_city, goal_city)
    print(f"UCS - Path: {path}, Cost: {cost}")

UCS - Path: ['A', 'C', 'E'], Cost: 5
```

OutPut:

```
UCS - Path: ['A', 'C', 'E'], Cost: 5
```